

HR Management System - Architecture Documentation

1. Project Structure (Architecture)

The **HR Management System** follows a **layered architecture** to ensure separation of concerns, maintainability, and scalability.

Technology Stack:

- **Backend:** REST API using ASP.NET 8
 - **Frontend:** ASP.NET MVC 8 with Razor View (CSHTML) and Bootstrap
 - **Database:** SQL Server (Entity Framework Core - Code First)
 - **CI/CD:** GitLab CI/CD setup
 - **Design Pattern:** Repository Pattern
-

2. Layered Architecture

1. HRMS.API (*Backend - RESTful API*)

- Exposes RESTful endpoints for the **HRMS.UI** frontend.
- Implements **service layer calls** to process business logic.
- Uses **Entity Framework Core Code-First** for database interactions.

2. HRMS.UI (*Frontend - MVC Application*)

- Implements the **ASP.NET MVC 8** framework.
- Communicates with **HRMS.API** via AJAX & REST API calls.
- Uses **Bootstrap** for UI components.

3. HRMS.DataAccess (*Database Layer*)

- **Entity Framework Core (EF Core) Code-First** approach.
- Manages database operations with **repository pattern**.
- Handles **migrations** for schema changes.

4. HRMS.Models (*Domain Models & DTOs*)

- Contains **business models** representing employees, job history, and other entities.
- Defines **Data Transfer Objects (DTOs)** for API responses.
- Ensures data consistency across layers.

5. HRMS.Services (*Service Layer - Business Logic*)

- Implements **business rules** and processing logic.
- Acts as an intermediary between **HRMS.API** and **HRMS.DataAccess**.
- Uses dependency injection to interact with repositories.

6. HRMS.Tests (*Unit Testing for Service Layer*)

- Implements **unit tests** for the **HRMS.Services** layer.
 - Uses **Mocking Frameworks** (like Moq) to isolate dependencies.
 - Ensures **business logic validation** without database dependency.
-

3. Key Design Considerations

- **Separation of Concerns (SoC):** Each layer has a distinct responsibility.
- **Scalability:** API and UI can be extended independently.
- **Testability:** The service layer is **unit-testable** with mock dependencies.
- **Maintainability:** Uses **Repository Pattern** to separate data access logic.
- **Modularity:** Each module can be **updated independently** without affecting others.

3. Diagram

